

Laborator 2

STRUCTURI ARITMETICE DE ÎNMULȚIRE

În acest laborator sunt studiate circuitele aritmetice de înmulțire utilizate în unitățile de calcul. Pe parcursul acestuia se vor prezenta structurile componente ale circuitelor aritmetice de înmulțire. Pentru exemplificare se vor prezenta structurile interne pentru circuitele de înmulțire care pot opera cu operanzi pe 4 biți.

1. Regiștrii paralel-paralel și serie-paralel cu deplasare stânga-dreapta

Regiștrii cu acces paralel și acces serie s-au tratat în cadrul altor discipline (vezi laboratorul 2 la disciplina calculatoare numerice). În cele ce urmează se prezintă o descriere foarte scurtă a acestora întrucât sunt necesari în construirea circuitelor de înmulțire. Se va prezenta la fiecare schema lui (la nivel de bistabili) și descrierea lui în cod VHDL urmărind modelul comportamental.

a. Registrul paralel-paralel

Este folosit pentru memorarea datelor de intrare în format paralel. Accesul la registru se face printr-un semnal de ceas care determină memorarea datelor de la intrare și regăsirea acestora pe ieșire.

Structura unui registru cu acces paralel, pe 4 biți, descrisă cu bistabili D este prezentată în figura de mai jos:

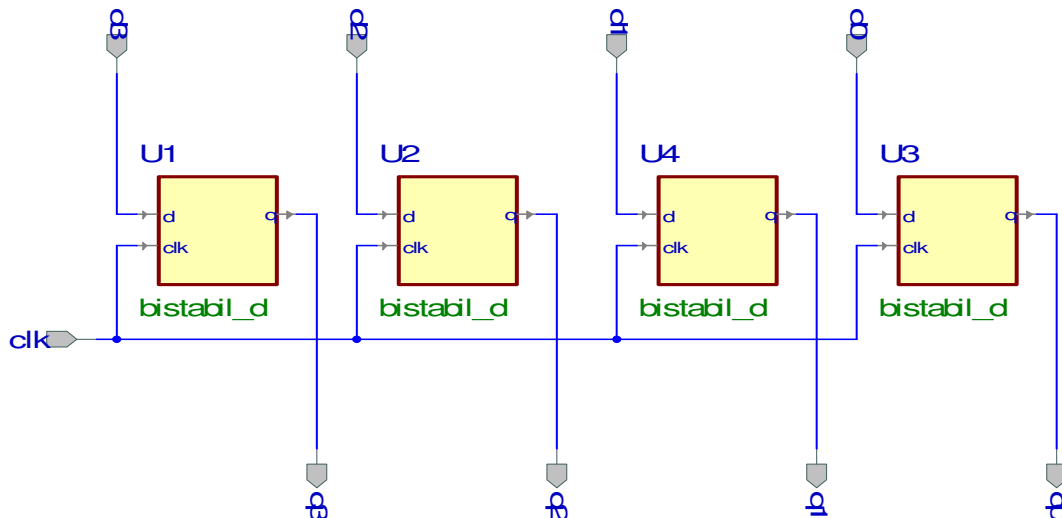


Fig. 1. Registrul paralel – paralel pe patru biți

Codul VHDL care descrie comportamentul registrului cu acces paralel, pe 4 biți, este prezentat în listingul de mai jos:

library IEEE;

```

use IEEE.STD_LOGIC_1164.all;

entity rpp4 is
    port(
        clk : in STD_LOGIC;
        d : in STD_LOGIC_VECTOR(3 downto 0);
        q : out STD_LOGIC_VECTOR(3 downto 0)
    );
end rpp4;

architecture rpp4_a of rpp4 is
begin

    process(d,clk)
        variable q_int : std_logic_vector(3 downto 0);
    begin
        if clk'event and clk='1' then
            q_int := d;
        end if;
        q <= q_int;
    end process;

end rpp4_a;

```

b. Registrul paralel cu deplasare serială (serie-paralel)

O asemenea structură, cu mici modificări, poate fi folosită la implementarea pentru diferite operații, de exemplu deplasarea. La schema la nivel de componente logice prezentată pentru registrul cu acces paralel în figura 1 se mai adaugă patru circuite de selecție care selectează sursa intrării D. În figura 2 este prezentată schema unui registru cu încărcare paralelă și deplasare serială la dreapta, pe 4 biți, realizat cu bistabili D. Circuitul poate opera în două moduri. Dacă pe intrarea „s_notp” primește valoarea 0 circuitul va opera la fel ca cel din figura 1, deoarece pe fiecare intrare D se va găsi unul din semnalele pi0..3. Dacă intrarea „s_notp” este 1 atunci pe intrarea D la primul bistabil (U1) se va conecta intrarea si în timp ce pe intrările D de la ceilalți bistabili se vor regăsi ieșirile bistabililor din etajele anterioare.

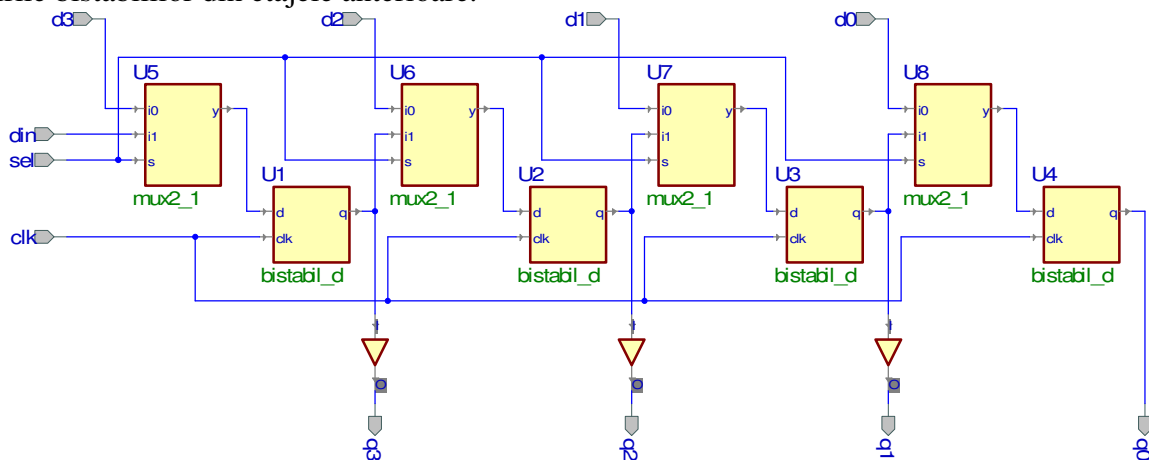


Fig. 2. Registrul paralel – serie cu deplasare la dreapta

În cel de-al doilea mod, în urma aplicării unui semnal de ceas, are loc deplasarea către dreapta, conform figurii, deci valoarea nouă a lui Q1 devine Q0. Valoarea nouă a lui Q0 este dată de intrarea „si”.

Codul VHDL care descrie comportamentul registrului cu acces paralel este prezentat în listingul de mai jos:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rps4 is
    port(
        si : in STD_LOGIC;
        s_notp : in STD_LOGIC;
        clk : in STD_LOGIC;
        pi : in STD_LOGIC_VECTOR(3 downto 0);
        q : out STD_LOGIC_VECTOR(3 downto 0)
    );
end rps4;

architecture rps4_a of rps4 is
    begin

        process(clk,pi,si,s_notp)
            variable q_int : std_logic_vector(3 downto 0);
            begin
                if clk'event and clk='1' then
                    if s_notp = '0' then
                        q_int := pi;
                    else
                        q_int(2 downto 0) := q_int(3 downto 1);
                        q_int(3) := si;
                    end if;
                end if;
                q <= q_int;
            end process;
        end rps4_a;
```

2. Circuitul de înmulțire secvențial pe 4 biți folosind al treilea algoritm de înmulțire

a. Al treilea algoritm de înmulțire

În figura de mai jos este prezentată organigrama pentru cel de-al treilea algoritm de înmulțire.

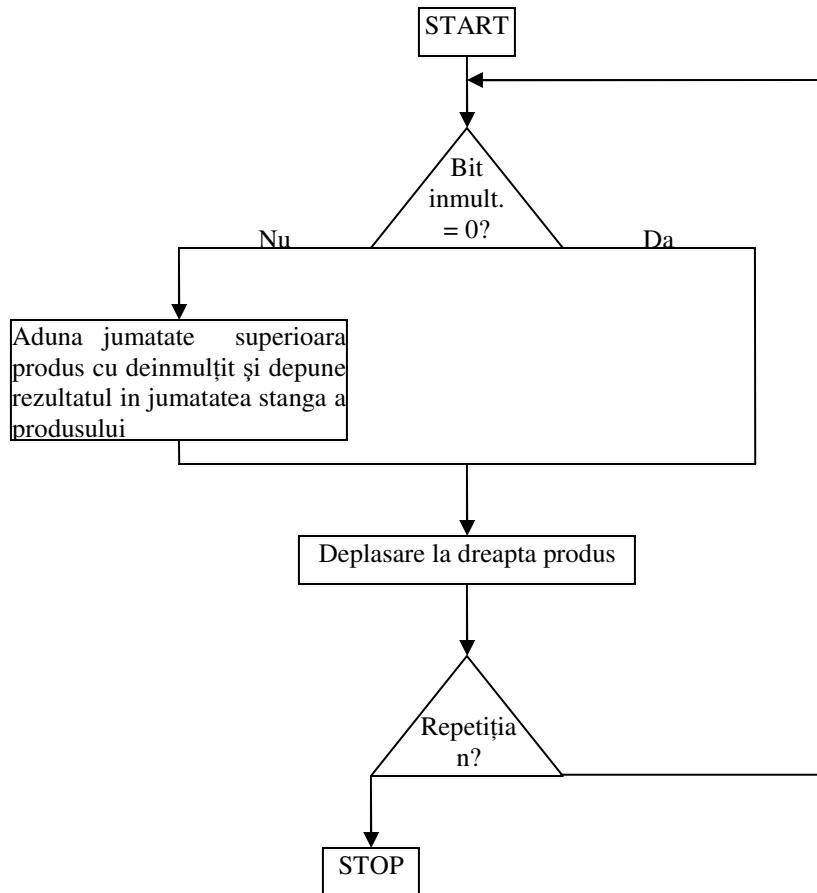


Fig. 3. Organigrama algoritmului 3 de înmulțire

Algoritmul începe cu testarea ultimului bit din înmulțitor. Dacă acest bit este 1 atunci jumătatea superioară a produsului este adunată cu deînmulțitul și rezultatul este depus în jumătatea superioară a produsului. Produsul este apoi deplasat la dreapta, împreună cu înmulțitorul. Pasul se repetă pentru fiecare bit din înmulțitor, deci dacă înmulțitorul este pe 4 biți atunci vor exista 4 pași distincți.

Avantajul acestui algoritm este acela că numărul de iterații (pași efectuați) este întotdeauna egal cu dimensiunea unuia din operanzi (împărțitorul) (nu depinde de valoarea operanzilor). Un alt avantaj îl constă faptul că împărțitorul se află depus în registrul produs, deci folosește resurse hardware limitate.

b. Circuitul de înmulțire

În componența unui circuit de înmulțire intră următoarele module:

- sumator 4 biți;
- registrul paralel-paralel pe 4 biți pentru deînmulțit;
- registrul serie-paralel cu deplasare la dreapta pe 8 biți – realizat, conform schemei de mai jos din doi regiștrii serie paralel pe 4 biți.

Toate aceste circuite au fost prezentate în detaliu în paragrafele anterioare sau pe parcursul laboratorului anterior. În figura de mai jos este prezentată schema circuitului de înmulțire folosind aceste trei module.

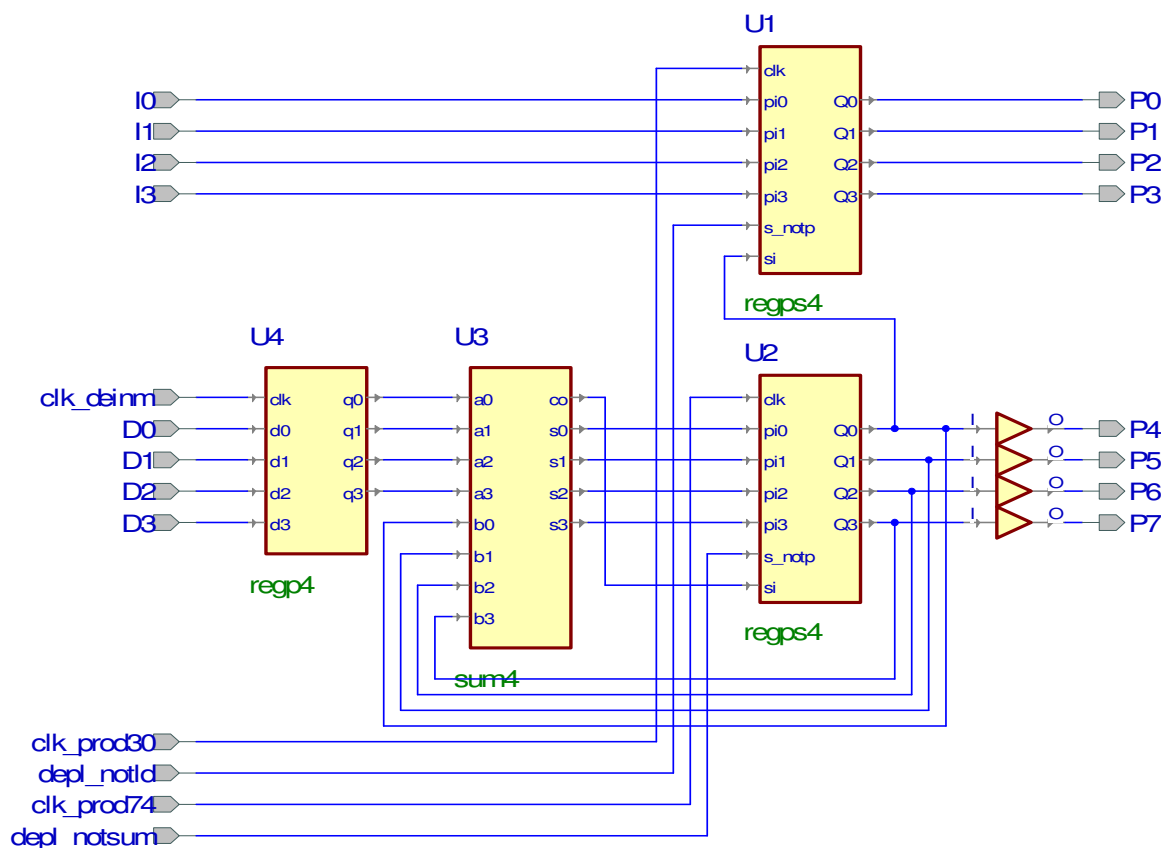


Fig. 4.Schema circuitului de înmulțire

Pinii introduși sunt următorii:

Denumire pini	Direcționalitate	Explicație
I3..I0	Intrări	Intrări înmulțitor
D3..D0	Intrări	Intrări de înmulțit
Clk_deinm	Intrare	Intrare ceas registru de înmulțit
Clk_prod30	Intrare	Intrare ceas registru înmulțitor – produs jumătate cea mai puțin semnificativă (cmps)
Depl_notld	Intrare	Intrare configurare registru produs jumătate cmps serie sau paralel
Clk_prod74	Intrare	Intrare ceas registru produs jumătate cea mai semnificativă (cms)
Depl_notsum	Intrare	Intrare configurare registru produs jumătate cms deplasare sau încărcare sumă
P7..P0	Ieșiri	Ieșiri produs. P0 joacă rol și de ieșire de test.

Urmărind algoritmul din figura 5 pașii parcurși într-o înmulțire completă sunt următorii:

- aplicarea pe intrările D3-D0 și I3-I0 a operanzilor pentru înmulțire (deînmulțit și înmulțitor);
- încărcarea operanzilor prin aplicarea unei tranziții 1-0 (aplicarea succesivă a valorilor 1 și 0) pe intrările clk_deinm și clk_prod30, cu totii registrii trecuti în modul paralel;
- inițializarea registrului care va conține jumătatea superioară a produsului cu valoarea 0. Inițializarea se poate face în mai multe moduri: fie prin încărcarea paralelă, după ce ne asigurăm că ieșirea sumatorului este 0, fie prin deplasare serială, după ce ne asigurăm că ieșirea de transport a sumatorului este 0. În acest al doilea caz este necesară aplicarea a 4 impulsuri de ceas pentru inițializarea cu 0 a fiecărui bit din registru.
- este testat pinul cel mai puțin semnificativ din împărțitor, în cazul de mai sus ieșirea P0;
- dacă valoarea acesteia este 1 atunci registrul U2 este trecut în modul paralel, prin trecerea intrării depl_notsum în 0, după care se încarcă valoarea de la ieșirea sumatorului prin aplicarea unei tranziții 0-1 pe intrarea clk_prod74;
- se efectuează deplasarea la dreapta a rezultatului: regiștrii U2 și U1 sunt trecuți în modul serial, deci se aplică pe intrările depl_notld și depl_notsum valorile 1, apoi se aplică ceas pentru deplasarea la dreapta întâi a conținutului registrului U2 (0-1 pe intrarea clk_prod74) și apoi a registrului U1 (0-1 pe intrarea clk_prod30);
- pașii 3,4 și 5 se repetă de 4 ori (numărul de biți ai înmulțitorului).

De precizat că starea inițială la registrul cu acces paralel și deplasare serială plasat la ieșirea sumatorului (circuitul U2 din figura 5) trebuie să fie 0.

Desfășurarea lucrării:

1. Se va proiecta un circuit de înmulțire a două numere pe patru octeți. Pentru aceasta se vor urmări pașii următorii:
 - a. Se va construi un proiect nou în ACTIVE HDL, și se va configura astfel încât să permită implementarea pe machetele de laborator cu Xilinx Spartan 3 (vezi anexa de laborator);
 - b. Se va proiecta un sumator pe patru biți (cu transport anticipat sau transport parțial) folosind cunoștințele cumulate din laboratorul anterior (subiect evaluare laborator anterior) – la construirea sumatorului se vor folosi porți logice din clasa Built In Symbols (vezi figura 5) sau descrierea acestuia pe componente în VHDL. În cazul în care sumatorul este deja descris în alt proiect fișierul cu descrierea acestuia și fișierele cu eventualele componente se pot include ca librării necompileate în proiectul nostru. În figura 6 sunt reamintite etapele necesare pentru includerea unui fișier cu o componentă deja descrisă.
 - c. Se vor construi regiștrii paralel – paralel și paralel – serie cu deplasare la dreapta conform figurilor 1 și 2 sau conform descrierii VHDL.
 - d. Folosind circuitul sumator proiectat la punctul b., registrul paralel – paralel și registrul serie paralel se va realiza schema circuitului de înmulțire a două numere pe patru biți (figura 5). Circuitele buffer necesare

pentru separarea ieșirilor se vor adăuga de asemenea din clasa Built In Symbols (fig. 8).

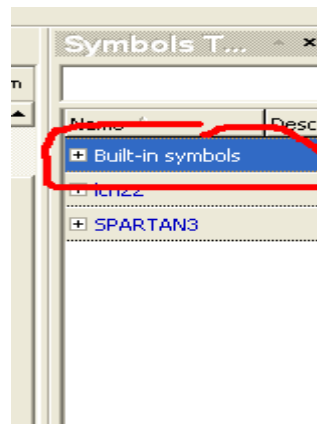
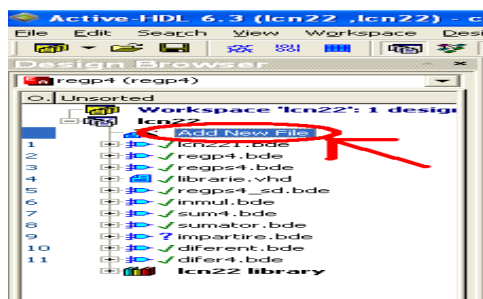
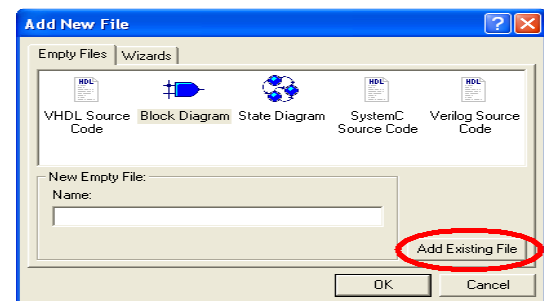


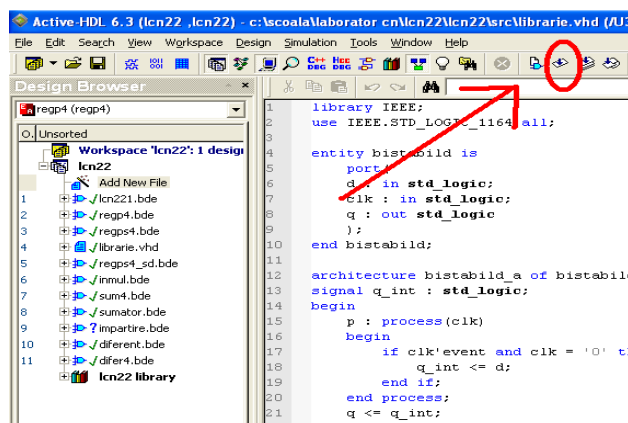
Fig. 5. Porțile logice elementare sau componentele declarate anterior se vor introduce din caseta Symbol Toolbox



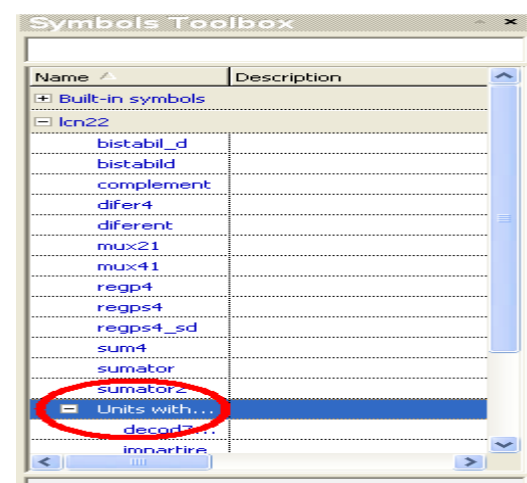
a. Se selectează „Add New File” din Design Browser



b. Se apasă pe „Add Existing File” și se caută calea către fișierul librărie



c. După ce fișierul librărie a fost atașat la proiect se deschide și se compilează



d. După compilare, componentele din librărie se găsesc în subclasa „Units without symbols” aflată în clasa cu numele proiectului.

Fig. 6. Etape în includerea unui fișier cu o descriere de componentă în proiect

- e. Pentru a verifica funcționalitatea circuitului, ieșirile acestuia se vor conecta la un decodor 7 segmente, cum se vede în figura de mai jos. Intrările cu înmulțitorul și deînmulțitul vor fi stabilite prin plasarea la valorile corespunzătoare în timp ce intrările de comenzi de tip clk se conectează la butoane prin interfețe (care asigura eliminarea oscilațiilor la comutarea butonului). Intrările care stabilesc direcționalitatea regiștrilor serie paralel vor fi conectate la comutatori (sw_dip).

Decoderul împreună cu circuitele de interfațare a butoanelor sunt descrise mai jos în VHDL.

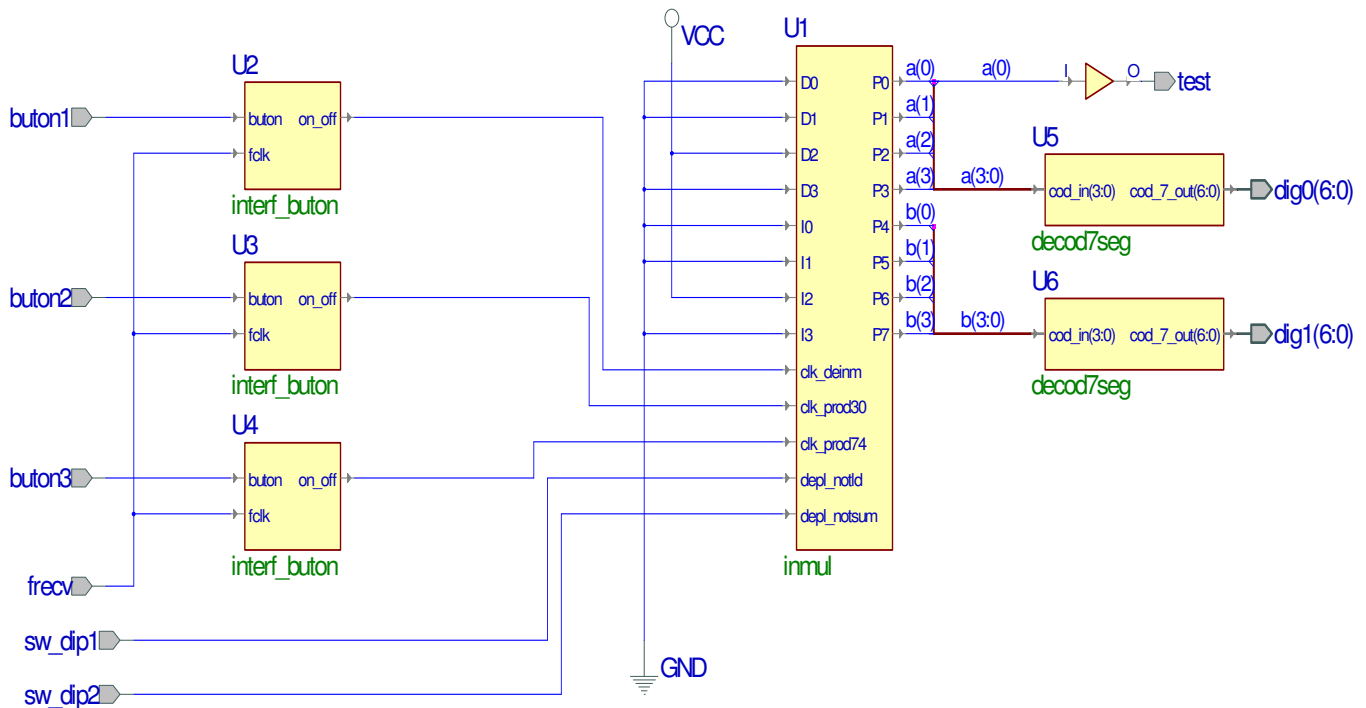


Fig. 10. Schema de testare a circuitului de înmulțire

```

Resetare registru P74
Incarcare deinmultit (buton1)
Incarcare inmultitor (swdip1 = 0, buton2)
Se repeta de 4 ori
{
  Test? (LED ON?)
  Daca da incarcare produs partial (swdip2 = 0. buton3)
  Deplasare produs P30 (swdip1 = 1, buton2)
  Deplasare produs P74 (swdip2 = 1, buton3)
}

```


Se va construi, sintetiza și implementa structura din figura 10 (se urmăresc pașii specificați în anexa). Pentru conectarea intrărilor și ieșirilor la butoanele respectiv afișoarele de pe machetă se vor realiza următoarele alocări de pini:

Pin alocat	Pin fizic
Buton1	D1
Buton2	C1
Buton3	B6
Sw_dip1	Y6
Sw_dip2	V6
Test	W2
Dig1(0)	C6
Dig1(1)	B8
Dig1(2)	E7
Dig1(3)	C5
Dig1(4)	E6
Dig1(5)	B5
Dig1(6)	A4
Dig0(0)	B10
Dig0(1)	A12
Dig0(2)	C10
Dig0(3)	A9
Dig0(4)	B9
Dig0(5)	A10
Dig0(6)	E9
Frecv	AA12

- f. Pentru operanzii introduși se vor parcurge pașii care realizează înmulțirea celor doi operanzi. Bitul de test (bcmpps din produs) este conectat separat ca ieșire la LED-ul 0 de pe machetă, acesta prezentând starea ultimului bit în funcție de care se va lua decizia în algoritm pentru încărcarea sumei.

Decodorul binar – 7 segmente:

```
---decodorul binar - 7 segmente
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity decod7seg is
  Port (      cod_in : in std_logic_vector(3 downto 0);
           cod_7_out: out std_logic_vector(6 downto 0)
  );
end decod7seg;
```

```

architecture decod7seg_a of decod7seg is
begin
    process(cod_in)
    begin
        case cod_in is
            when "0000" => cod_7_out<= "0111111";
            when "0001" => cod_7_out <= "0000110";
            when "0010" => cod_7_out <= "1011011";
            when "0011" => cod_7_out <= "1001111";
            when "0100" => cod_7_out <= "1100110";
            when "0101" => cod_7_out <= "1101101";
            when "0110" => cod_7_out <= "1111101";
            when "0111" => cod_7_out <= "0000111";
            when "1000" => cod_7_out <= "1111111";
            when "1001" => cod_7_out <= "1101111";
            when "1010" => cod_7_out <= "1110111";
            when "1011" => cod_7_out <= "1111100";
            when "1100" => cod_7_out <= "0111001";
            when "1101" => cod_7_out <= "1011110";
            when "1110" => cod_7_out <= "1111001";
            when "1111" => cod_7_out <= "1110001";
            when others => cod_7_out <= "0000000";
        end case;
    end process;
end decod7seg_a;

```

Circuit interfață butoane (evitare debouncing):

```

----circuit interfata butoane
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity interf_buton is
    Port (
        fclk : in std_logic;
        buton : in std_logic;
        on_off: out std_logic
    );
end interf_buton;

architecture interf_buton_a of interf_buton is
    signal decnt : std_logic_vector(17 downto 0);
    signal onoff : std_logic;
begin
    --proces folosit pentru debouncing- interval de 5ms de temporizare
    deb : process (fclk,buton)
    begin
        if buton = '1' then
            decnt <= "000000000000000000";
            onoff <= '1';
        elsif fclk'event and fclk = '1' then
            if decnt < 250000 then
                decnt <= decnt + 1;
                onoff <= '1';
            else

```

```

                                onoff <= '0';
                                end if;
                                end if;
                                end process;
                                on_off <= onoff;
                                end interf_buton_a;

```

Probleme. Întrebări:

1. Care este numărul de pași necesari pentru înmulțirea a două numere pe 4 octeți fiecare folosind al treilea algoritm de împărțire?
2. Ce se întâmplă în situația în care, la circuitul de înmulțire pe 4 biți din figura 4, apare o depășire la una din adunări, într-un pas?
3. Cum se poate reseta registrul în care se află stocată jumătatea cea mai semnificativă a produsului?
4. Să se proiecteze un registru paralel-paralel pe 8 biți.
5. Să se proiecteze un registru serie-paralel pe 8 biți.
6. Să se proiecteze un circuit de înmulțire pe 8 biți.

Grilă de evaluare (proiectare):

Activitate	Punctaj
Proiectare sumator (diferență) 4 biți	2
Proiectare registru paralel-paralel	1
Proiectare registru serie-paralel	1
Proiectare circuit înmulțire + schemă test	3
Implementare circuit test	3